



Análisis del dataset de Títulos de Netflix

Datos, limpieza y modelos en Python

Este notebook usa el dataset clásico del **Netflix** para practicar un flujo completo de trabajo con datos:

- Cargar y entender una tabla
- Detectar problemas de calidad
- Limpiar valores faltantes y tipos incorrectos
- Crear variables nuevas a partir de la información existente

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
plt.rcParams["figure.figsize"] = (8, 5)
plt.rcParams["axes.grid"] = True
```

1. Carga de datos

Cada fila representa una película o una serie que se ofrece en el catalogo de Netflix. Las columnas más importantes son:

- **Type:** Esquema de contenido (**Movie** o **TV Show**)
- **Title:** Nombre de la película o serie
- **Director:** Nombre del director de la película o serie
- **Cast:** Nombre de los actores principales
- **Country:** País de origen
- **date_added:** Fecha en la que se agregó a la plataforma

- **release_year:** Año de estreno
- **Rating:** Certificación de contenido (**G** , **PG** , **PG-13** , **R** , etc...)
- **Duration:** Tiempo estimado en minutos o en temporadas
- **Listed_in:** Categoría en la cual se lista el contenido
- **Description:** Breve reseña de la historia de la película o Serie

Usaremos un CSV local en `netflix_titles.csv` .

Código:

```
from pathlib import Path

REPO_ROOT = Path.cwd() if (Path.cwd() / "data").exists() else Path.cwd().parent
DATA_PATH = REPO_ROOT / "data" / "netflix_titles.csv"
```

```
if DATA_PATH.exists():
    df_original = pd.read_csv(DATA_PATH)
    print("Filas y columnas originales:", df_original.shape)
    df_original.head()
else:
    print("El archivo no existe en el disco")
```

Output:

```
Filas y columnas originales: (8807, 12)
```

2. Revisión inicial de los datos

Antes de limpiar, siempre conviene responder tres preguntas básicas:

1. ¿Qué columnas tengo?
2. ¿Qué tipo de dato tiene cada una?
3. ¿Hay valores faltantes o algo que no cuadre?

En este dataset ya podemos anticipar un problema típico: **muchas columnas tienen nulos**, especialmente `Director` y `Country`.

Código:

```
print("Columnas disponibles:")
print(df_original.columns.tolist())

print("\nTipos de datos:")
display(df_original.dtypes.to_frame("tipo"))

print("\nValores faltantes por columna:")
missing = df_original.isnull().sum().sort_values(ascending=False)
missing_pct = (missing / len(df_original) * 100).round(1)
display(pd.DataFrame({"faltantes": missing, "porcentaje": missing_pct}))

print("\nResumen numérico:")
display(df_original.describe().T)

print("\nCantidad de Series y Películas:")
display(df_original["type"].value_counts().rename({0: "Movie", 1: "TV Show"}))
```

Output: 🔗

Columnas disponibles:

['show_id', 'type', 'title', 'director', 'cast', 'country', 'date_added', 'release_year', 'rating',

Tipos de datos:

show_id	str
type	str
title	str
director	str
cast	str
country	str
date_added	str
release_year	int64
rating	str
duration	str
listed in	str

description str

Valores faltantes por columna:

	faltantes	porcentaje
director	2634	29.9
country	831	9.4
cast	825	9.4
date_added	10	0.1
rating	4	0.0
duration	3	0.0

Cantidad de Series y Películas:

```
type
Movie      6131
TV Show    2676
Name: count, dtype: int64
```

3. Auditoría de calidad

La limpieza empieza con un diagnóstico. Revisamos:

1. **Duplicados:** filas repetidas que podrían inflar conteos
2. **Valores imposibles:** edades negativas, tarifas negativas, clases fuera de rango
3. **Cardinalidad:** cuántos valores distintos tiene cada columna categórica
4. **Consistencia:** si los tipos de datos coinciden con el significado de la columna

Esta auditoría nos dice qué hay que corregir y por qué.

Código:

```
print("--- Auditoría de calidad ---")

duplicados = df_original.duplicated().sum()
print("Filas duplicadas:", duplicados)

print("\nAño de lanzamiento negativos:", (df_original["release_year"] < 0).sum())

print("Tipo de contenido invalido:", (~df_original["type"].isin(["Movie", "TV Show"])).sum())

print("\nValores únicos por columna:")
for col in df_original.columns:
    print(f"- {col}: {df_original[col].nunique()}")

print("\nDistribución de Peliculas por su rating:")
display(df_original["rating"].value_counts())
```

Output:

```
--- Auditoría de calidad ---  
Filas duplicadas: 0  
  
Año de lanzamiento negativos: 0  
Tipo de contenido invalido: 0  
  
Valores únicos por columna:  
- show_id: 8807  
- type: 2  
- title: 8807  
- director: 4528  
- cast: 7692  
- country: 748  
- date_added: 1767  
- release_year: 74  
- rating: 17  
- duration: 220  
- listed_in: 514  
- description: 8775
```

Distribución de Peliculas por su rating:

```
rating  
TV-MA      3207  
TV-14      2160  
TV-PG      863  
R          799  
PG-13      490  
TV-Y7      334  
TV-Y       307  
PG         287  
TV-G       220  
NR         80  
G          41  
TV-Y7-FV   6  
NC-17      3  
UR         3  
74 min     1  
84 min     1  
66 min     1  
Name: count, dtype: int64
```

Visualización:

4. Limpieza de datos

Decisiones de limpieza

1. **Director:** faltan 2634 valores → Los reemplazamos por **Desconocido**
2. **Country:** faltan 831 valores → Los reemplazamos por **Desconocido**
3. **Cast:** faltan 825 valores → Los reemplazamos por **Desconocido**
4. **Date_added:** faltan 10 valores → Filas eliminadas
5. **Rating:** faltan 4 valores → Agregados manualmente
6. **Duration:** faltan 3 valores → Filas eliminadas

Código:

```
print("--- Iniciando limpieza de datos ---")

df = df_original.copy()
filas_inicio = len(df)

# 1) Director: faltan 2634 valores. Los reemplazamos por la la palabra(Desconocido).
df["director"] = df["director"].fillna("Desconocido")

# 2) Country: faltan 831 valores. En lugar de borrar esas filas, Los reemplazamos por la la palabra(Desconocido).
df["country"] = df["country"].fillna("Desconocido")

# 3) Cast: faltan 825 valores. En lugar de borrar esas filas, Los reemplazamos por la la palabra(Desconocido).
df["cast"] = df["cast"].fillna("Desconocido")

# 4) date_added: faltan 10 valores. por lo cual seran eliminados
df = df.dropna(subset=["date_added"])

# 5) Encontrar los registros que no tienen rating
display(df[df["rating"].isnull()])
```

```
#Asignar el rating con base a su titulo
df.loc[df["title"] == "13TH: A Conversation with Oprah Winfrey & Ava DuVernay", "rating"] = "TV-14"

df.loc[df["title"] == "Gargantia on the Verdurous Planet", "rating"] = "TV-14"

df.loc[df["title"] == "Little Lunch", "rating"] = "TV-G"

df.loc[df["title"] == "My Honor Was Loyalty", "rating"] = "TV-MA"

#Verificar que no existan valores faltantes
print("Ratings faltantes:", df["rating"].isnull().sum())

# 6) duration: faltan 3 valores. por lo cual seran eliminados
df = df.dropna(subset=["duration"])

#Verificar los valores faltantes por columna
print("
Valores faltantes por columna despues de realizar la limpieza:")
missing = df.isnull().sum().sort_values(ascending=False)
missing_pct = (missing / len(df) * 100).round(1)
display(pd.DataFrame({"faltantes": missing, "porcentaje": missing_pct}))
```

Output:

```
--- Iniciando limpieza de datos ---

show_id      type  title  director      cast  country      date_added      release_year  rating duration      listed_in  descri
5989   s5990  Movie  13TH: A Conversation with Oprah Winfrey & Ava ...  Desconocido  Oprah Winfrey, Ava DuVernay  Descon
6827   s6828  TV Show      Gargantia on the Verdurous Planet Desconocido  Kaito Ishikawa, Hisako Kanemoto, Ai Kayano, Ka...
7312   s7313  TV Show      Little Lunch Desconocido  Flynn Curry, Olivia Deeble, Madison Lu, Oisín ...  Australia  Februa
7537   s7538  Movie  My Honor Was Loyalty Alessandro Pepe      Leone Frisa, Paolo Vaccarino, Francesco Miglio...  Italy  March

Ratings faltantes: 0

Valores faltantes por columna despues de realizar la limpieza:

faltantes      porcentaje
show_id      0      0.0
type  0      0.0
title  0      0.0
director      0      0.0
cast  0      0.0
country      0      0.0
date_added      0      0.0
```


5. Creación de variables nuevas

Muchas veces la información útil no viene lista en una sola columna. Hay que **construirla**.

Crearemos:

- **content_age**: Antigüedad del contenido (2026 - release_year)
- **decade**: Década de lanzamiento (1980s , 1990s , 2000s , 2010s , 2020s)
- **month_added**: Mes en el que el contenido fue agregado

Estas variables ayudan a responder preguntas como:

- ¿Netflix agrega principalmente contenido reciente o antiguo?
- ¿Qué décadas tienen más contenido disponible?
- ¿En qué meses Netflix agrega más títulos a su catálogo?
- ¿Cuál es la antigüedad promedio del contenido disponible?

Código:

```
# Antigüedad del contenido.
df["content_age"] = 2026 - df["release_year"]

# Década de lanzamiento.
df["decade"] = (df["release_year"] // 10) * 10
df["decade"] = df["decade"].astype(str) + "s"

# Convertir la fecha a formato datetime.
df["date_added"] = df["date_added"].str.strip()
df["date_added"] = pd.to_datetime(df["date_added"])

# Mes en el que el contenido fue agregado.
df["month_added"] = df["date_added"].dt.month_name()

new_columns = [
    "content_age",
    "decade",
    "month_added"
]
```

```
display(df[new_columns].head(10))

print("
Contenido por década:")
display(df["decade"].value_counts().sort_index())

print("
Contenido agregado por mes:")
display(df["month_added"].value_counts())

print("
Antigüedad del contenido:")
display(df["content_age"].describe())
```

Output:

content_age	decade	month_added
0	6	2020s September
1	5	2020s September
2	5	2020s September
3	5	2020s September
4	5	2020s September
5	5	2020s September
6	5	2020s September
7	33	1990s September
8	5	2020s September
9	5	2020s September

Contenido por década:

decade	
1920s	1
1940s	15
1950s	11
1960s	25
1970s	70
1980s	129
1990s	274
2000s	807

6. Visualizaciones exploratorias

Las tablas resumen números, pero las gráficas ayudan a ver patrones más rápido

- 1. La diferencia de lanzamiento de películas y series por países,
- 2. cómo se comporta netflix al agregar nuevos títulos a lo largo de un año,
- 3. la cantidad de películas y series en función de la década,

Código:

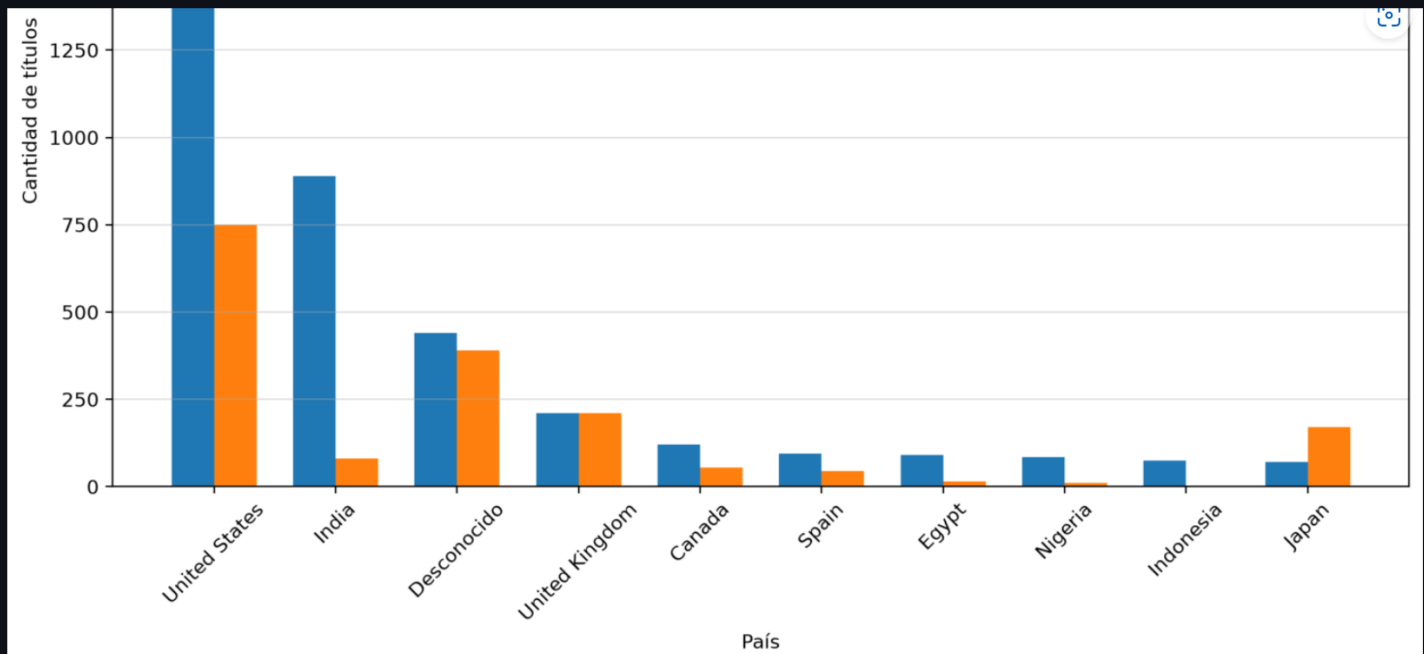
```
print("--- Películas y series por país ---")

# Películas y series por país (Top 10)

country_type = (
    df.groupby(["country", "type"])
      .size()
      .unstack(fill_value=0)
      .sort_values(by="Movie", ascending=False)
      .head(10)
)

country_type.plot(
    kind="bar",
    figsize=(10,6),
    title="Cantidad de películas y series por país (Top 10)"
)

plt.ylabel("Cantidad de títulos")
plt.xlabel("País")
plt.xticks(rotation=45)
plt.show()
```



Código:

```
print("--- Cantidad de títulos agregados por mes ---")

month_order = [
    "January", "February", "March", "April",
    "May", "June", "July", "August",
    "September", "October", "November", "December"
]

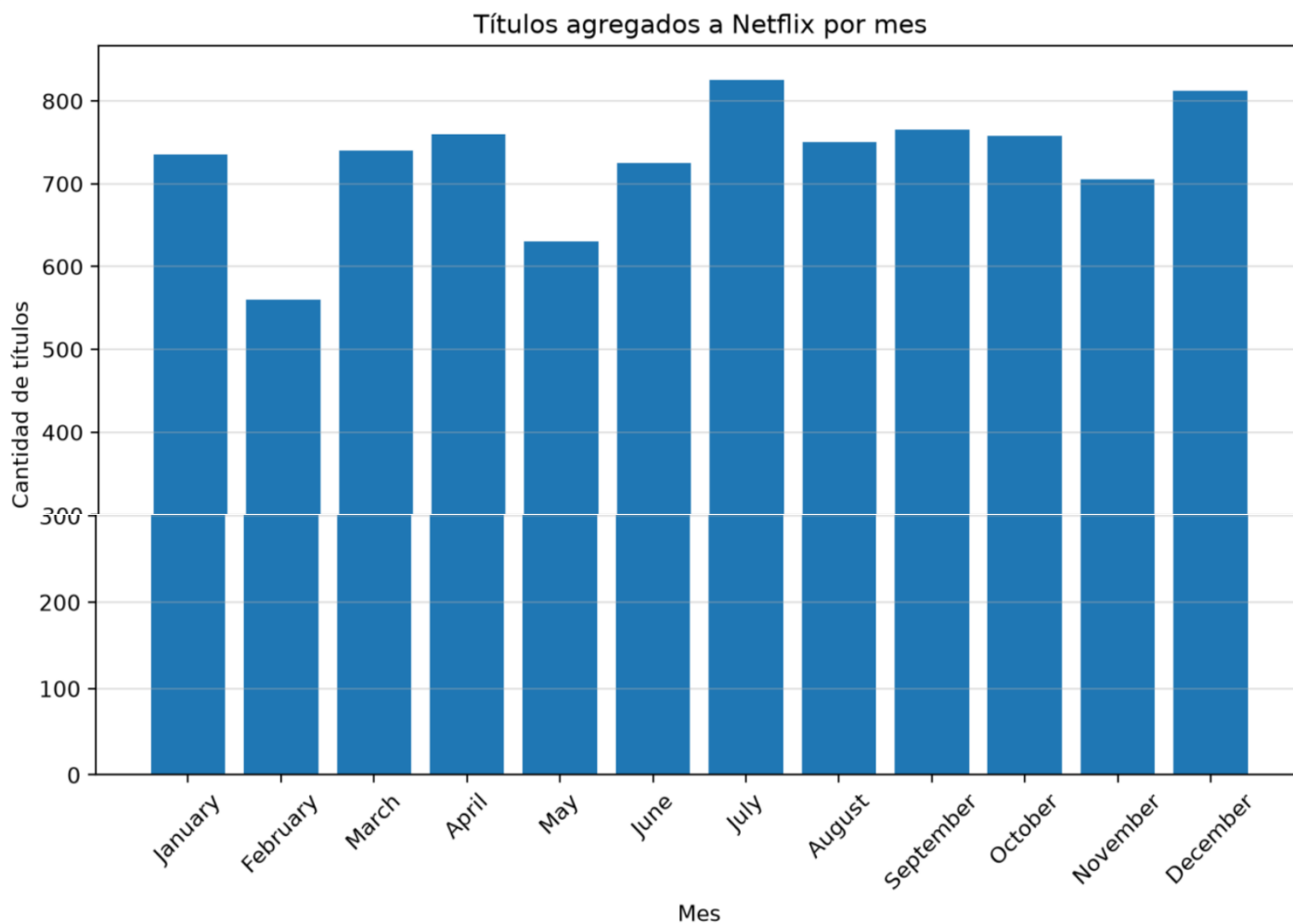
titles_month = (
    df["month_added"]
    .value_counts()
    .reindex(month_order)
)

titles_month.plot(
    kind="bar",
```

```
    title="Títulos agregados a Netflix por mes"
)

plt.ylabel("Cantidad de títulos")
plt.xlabel("Mes")
plt.xticks(rotation=45)
plt.show()
```

Títulos agregados a Netflix por mes



Código:

```
print("--- Cantidad de películas y series por década ---")

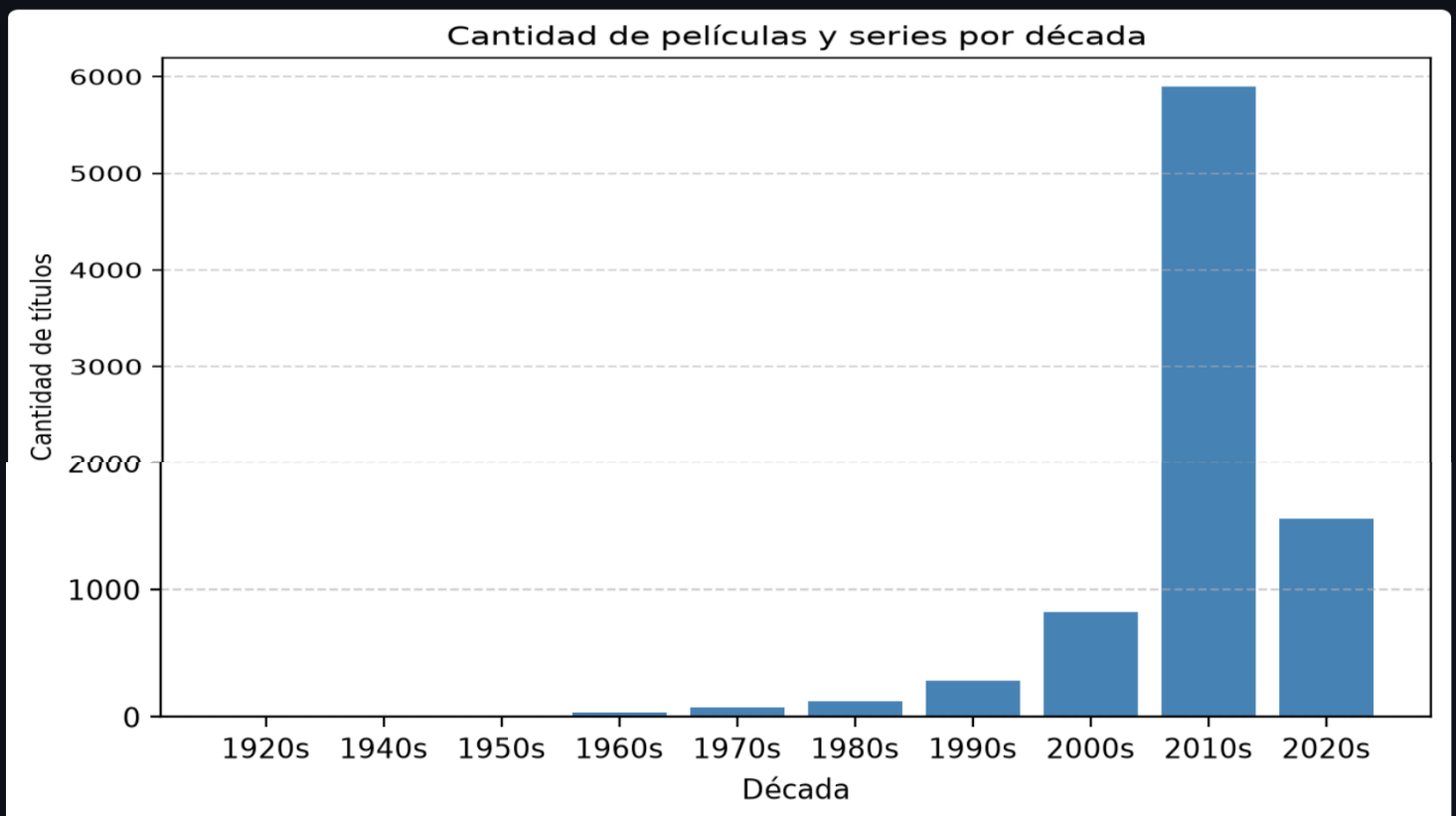
# Cantidad de títulos por década

titles_decade = (
    df["decade"]
    .value_counts()
    .sort_index()
)

titles_decade.plot(
    kind="bar",
    title="Cantidad de películas y series por década"
)

plt.ylabel("Cantidad de títulos")
plt.xlabel("Década")
plt.xticks(rotation=0)
plt.show()
```

Cantidad de películas y series por década



Código:

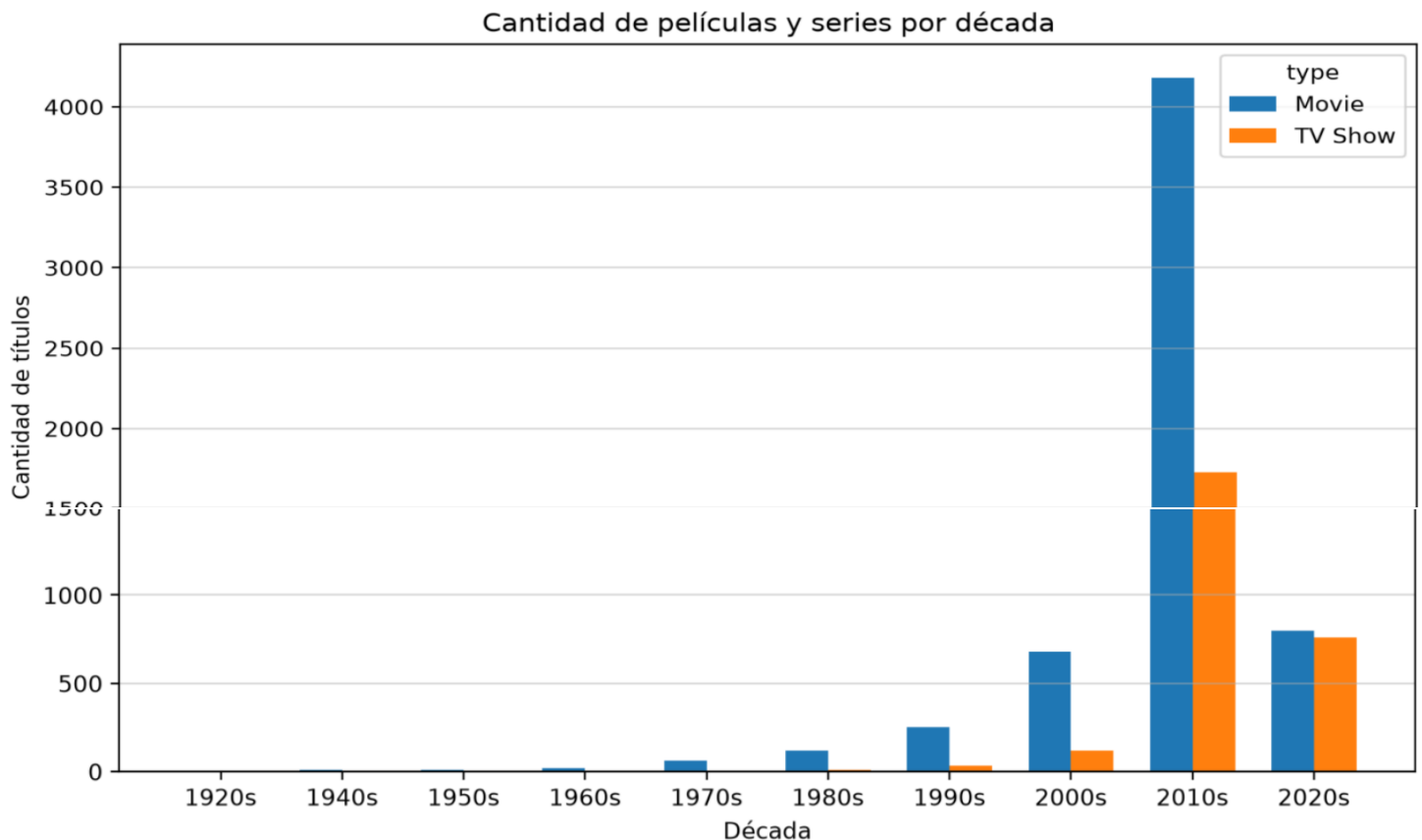
```
# Películas y series por década

decade_type = (
    df.groupby(["decade", "type"])
      .size()
      .unstack(fill_value=0)
)

decade_type.plot(
    kind="bar",
    figsize=(10,6),
    title="Cantidad de películas y series por década"
)

plt.ylabel("Cantidad de títulos")
plt.xlabel("Década")
plt.xticks(rotation=0)
plt.show()
```

Cantidad de películas y series por década



Cómo leer estas visualizaciones

- 1. **Country:** En la grafica se puede ver que el pais de origen de contenido mas representativo es Estados unidos
- 2. **Month_added:** Se puede ver que en julio y diciembre existe un mayor numero de titulos agregados por la plataforma aunque en todo el año se mantiene uniforme
- 3. **Decade:** La gran mayoría de las peliculas y series que se encuentran disponibles en netflix son de la decada de 2010

7 Regresión logística para encontrar la probabilidad de que el nuevo contenido sea pelicula

Después de limpiar y crear variables, podemos modelar la pregunta central:

¿El año de lanzamiento influye en la probabilidad de que un contenido sea una película o una serie?

Usamos regresión logística porque type se puede modelar como una variable binaria (0: TV Show o 1: Movie).

Variables del modelo:

release_year country

La fórmula con statsmodels nos permite interpretar coeficientes.

Código:

```
print("--- Regresión logística---")

df["type"] = df["type"].map({
    "Movie": 1,
    "TV Show": 0
})

modelo_df = df[[
    "type", "release_year"
]].copy()

formula = "type ~ release_year "
modelo_logit = smf.logit(formula, data=modelo_df).fit(dis=0)

display(modelo_logit.summary().tables[1])
print("Pseudo R-cuadrado:", round(modelo_logit.prsquared, 4))
```


Output:

--- Regresión logística para encontrar la probabilidad de que el contenido sea una película ---

	coef	stderr	z	P> z	[0.025	0.975]
Intercept	162.6424	9.899	16.431	0.000	143.242	182.043
release_year	-0.0803	0.005	-16.354	0.000	-0.090	-0.071

Pseudo R-cuadrado: 0.0381

****Interpretación del modelo logístico****

En regresión logística, un coeficiente positivo indica que aumenta la probabilidad del evento (El contenido sea película). Un coe

Ejemplos típicos en este dataset:

release_year positivo → A mayor año de lanzamiento, mayor probabilidad de que el contenido sea Movie.

El ****Pseudo R-cuadrado**** da una idea general del ajuste, pero no se interpreta igual que en regresión lineal.

8 Evaluación con train/test split

Para revisar si el modelo generaliza, separamos datos de entrenamiento y prueba.

Esto es importante en cualquier curso de datos: no basta con ajustar bien en la misma muestra; hay que probar con datos que el modelo no vio durante el entrenamiento

Código: ↻

```
# Preparar matriz para sklearn usando las mismas variables.
X = pd.get_dummies(modelo_df.drop(columns=["type"]), drop_first=True)
y = modelo_df["type"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Ajustar con statsmodels en entrenamiento.
train_data = X_train.copy()
train_data["type"] = y_train.values

formula_cols = "type ~ " + " + ".join([c for c in train_data.columns if c != "type"])
modelo_train = smf.logit(formula_cols, data=train_data).fit(dis=0)

# Predecir probabilidades y clases en prueba.
probs = modelo_train.predict(X_test)
preds = (probs >= 0.5).astype(int)
```

```
# Predecir probabilidades y clases en prueba.
probs = modelo_train.predict(X_test)
preds = (probs >= 0.5).astype(int)

print("Exactitud en prueba:", round(accuracy_score(y_test, preds), 4))
print("
Matriz de confusión:")
display(pd.DataFrame(
    confusion_matrix(y_test, preds),
    index=["Real: No", "Real: Sí"],
    columns=["Pred: No", "Pred: Sí"]
))

print("
Reporte de clasificación:")
print(classification_report(y_test, preds, target_names=["TV Show", "Movie"]))
```

weighted avg	0.49	0.70	0.57	1759
--------------	------	------	------	------